Combining Z and Temporal Interval Logics for the Formalization of Properties and Behaviors of Embedded Systems

Robert Büssow and Wolfgang Grieskamp

Technische Universität Berlin,
Institut für Kommunikations- und Softwaretechnik
Sekr. FR5-13, Franklinstr. 28/29, D-10587 Berlin, Germany
E-mail: {buessow,wg}@cs.tu-berlin.de

Abstract. We describe a variant of discrete temporal interval logics which is embedded, and extends the Z notation. The resulting formalism, called $\mathcal{D}Z$, includes the usual set of operators known from interval logics together with a set of operators tailored for the operational description of deterministic process behavior. We apply our approach to the foundation of a combination of Statecharts and Z by giving a translation from a significant subset of Statecharts into $\mathcal{D}Z$.

1 Introduction

Applying formal techniques to the development of safety-critical embedded systems is promising, since the proper function of these systems is important enough to be worth possible extra effort induced by a formal approach. A method for applying formal techniques for embedded systems is currently developed in the context of our working area, the ESPRESS project[1]: a specification of a safety-critical system consists of a *model* described by a combination of Statecharts and Z and a set of *properties* (usual safety-critical requirements), which have to be a consequence of the model.

In this paper, we describe $\mathcal{D}Z$ ("Dynamic Z"), a moderate extension of Z by discrete temporal interval logics, which shall serve as a logical framework for a methodological approach as sketched above. $\mathcal{D}Z$ provides the usual set of operators known from interval logics together with operators tailored for the operational description of deterministic process behavior. Due to the combination of these operators with temporal interval logics, we get a framework in which behavior and properties of embedded systems can be described simultaneously.

[1] The ESPRESS project is a cooperation of industry and research institutes funded by the German ministry BMBF.

An axiomatic framework based on temporal logics does not obviously support *concurrency*. We view concurrency conceptually as a situation where distinguishable *activities* compete for the access to restricted *resources* (let it be shared variables, channels, or whatsoever). Activities and resources can be modeled in $\mathcal{DZ}$ without a need for extending the language. We will show this for the case of shared variables with "racing".

We apply our approach by providing a sketch of a translation from a significant subset of Statemate-Statecharts into $\mathcal{DZ}$. The translation serves as a formalization of the semantics of this version of Statecharts; even more important, it defines a formal basis for relating Statecharts with temporal properties, and – in a next step, which we will not be tackled in this paper – for combining Statecharts and Z as a model oriented specification language as it is used in ESPRESS [2,1].

2 Integrating Temporal Interval Logics and Z

Z is a relative successful specification formalism based on set-theory and first order logic (cf. [15]). One restriction of Z is that though it is commonly used for the specification of stateful systems, it does not provide a mechanism for directly specifying invariants on *traces* of states. Consider, for example, an embedded system where we want to specify the invariant that "when the system passes state S, it will eventually pass either state S_1 or state S_2". This kind of property cannot be directly specified; rather, in the operational style of Z one has to ensure it "by construction".

By searching for concepts which fit to Z and which enable the descriptive specification of state traces, one naturally comes to some version of temporal logics. For $\mathcal{DZ}$, we have chosen *discrete temporal interval logics* [13,3]. In this logics, a property such as the above can be formulated as $\mathsf{everywhere}\ (S \chi \mathit{true} \Rightarrow S \chi \mathit{true} \chi S_1 \vee S \chi \mathit{true} \chi S_2)$. This formula looks similar to a regular expression or non-deterministic automaton, a concept software-engineers are used to. Moreover, interval logics allows to model not only properties but also operations by the finite traces they produce (for Z operations that would be traces of length 2).

2.1 Embedding of DZ in Z

$\mathcal{DZ}$ adds to Z the concepts of *dynamic values* and *dynamic schemata*. A dynamic value models a trace of elements of some type, and can be represented directly in Z. A dynamic schema bundles a collection of dynamic values which simultaneously satisfy a property given in temporal interval logics. Let as consider an example: the dynamic schema

$S \mathbin{\mathbf{b}} [\, x, y : \mathrm{dyn}\, \boldsymbol{\upsilon} \mid (\lceil x > 0 \rceil \boldsymbol{\chi} \lceil x < 0 \rceil) \wedge \lceil y \geq x \rceil \,]$ describes those dynamic values x and y – traces of integers – such that x is greater 0 up-to some state and less 0 after this state, whereas y is in each state greater or equal x. Hereby, $\mathrm{dyn}\, A$ is a normal Z set-constructor which denotes the set of traces of elements from some set A. For each binding $\sigma \in S$, $\sigma.x$ and $\sigma.y$ represent traces which are constraint as described above. By definition, the traces of the components of a binding of a dynamic schema have the same length.

Since dynamic schemata denote plain sets of bindings, they can be used wherever plain schemata can. For example, the schema expression $S \wedge [x, y : \mathrm{dyn}\, \boldsymbol{\upsilon} \mid x = y]$ conjuncts the dynamic schema S with a plain schema, which enforces the dynamic values x and y to be equal as a whole, meaning pointwise equality of their traces. $\mathcal{D}Z$ allows to declare "types" of dynamic values. It therefore introduces one further construct, a dynamic set comprehension. For example, the set $X == \{x : \mathrm{dyn}\, \boldsymbol{\upsilon} \mid \lceil x > 0 \rceil \boldsymbol{\chi} \lceil x < 0 \rceil\}$ denotes a subset of $\mathrm{dyn}\, \boldsymbol{\upsilon}$ where traces behave as described for the example above. With this definition of X, an alternate description for the above defined dynamic schema S would be $[\, x : X;\ y : \mathrm{dyn}\, \boldsymbol{\upsilon} \mid \lceil y \geq x \rceil \,]$. This again is equivalent to $[\, x, y : \mathrm{dyn}\, \boldsymbol{\upsilon} \mid x \in X \wedge \lceil y \geq x \rceil \,]$, as to be expected in a Z based framework.

2.2 Temporal Predicates of $D\mathbf{Z}$

Temporal predicates are a superset of normal Z predicates: element test, conjunction, negation, universal quantification, and its derivatives.

State Observations. The *state* predicate $\lceil \phi \rceil$ is satisfied for those traces where the normal Z predicate ϕ is satisfied at each point. For example, $\lceil x > 0 \rceil$ holds for traces of x such that for every point i in the trace, $x\, i > 0$ holds (representing the trace of x as a function $\boldsymbol{\gamma}\, \boldsymbol{\kappa}\, \boldsymbol{\upsilon}$). In a state predicate a variable $x : \mathrm{dyn}\, A$ denotes an element of type A. A second form of observing states is the *transition* predicate, which allows to relate observations at the beginning and the end of a trace: $\boldsymbol{\Psi} x' = x + 1\, \boldsymbol{\Omega}$ describes those traces, where in the last state the value of x is by one greater than in the first.

Length Constraints. A *length constraint* is denoted by the predicate $\ell\ \underline{R}\ e$, where e must denote a natural number, and $\underline{R}$ is some binary relation on natural numbers. For example, $\ell \leq 2$ describes those traces which length is less or equal 2. A special form of a length constraint is denoted as $\ell = \infty$, which holds for all traces of infinite length. For convenience, we can also annotate length constraints at other temporal predicates: $\lceil x \leq 0 \rceil_{=1}$ abbreviates $\lceil x \leq 0 \rceil \wedge \ell = 1$.

Chopping. The basic chopping operator of $\mathcal{D}Z$ is *overlapping chop*, $\delta_1 \boldsymbol{\tau} \delta_2$. It is satisfied for those traces which can be chopped at a state such that δ_1 holds up to this state and δ_2 holds from this state up-to the end. For example, $\lceil x > 0 \rceil \boldsymbol{\tau} \boldsymbol{\Psi} x' = x - 1 \, \underline{\boldsymbol{\Omega}}_2$ describes those traces where x is constrained to be greater than zero in any state except the last, and in the last state its value is decremented by one compared to the previous state. The overlapping chop is actually our basic chop operator, and the normal chop known from interval logics, $\delta_1 \boldsymbol{\chi} \delta_2$, is a derived form, defined as $\delta_1 \boldsymbol{\tau} \mathit{true}_{=2} \boldsymbol{\tau} \delta_2$.

Quantification. $\mathcal{D}Z$ entails the usual versions of existential and universal quantification of interval logics. For example, $\mathsf{everywhere}\ (\ell > 1 \Rightarrow \boldsymbol{\Psi} x < x' \, \boldsymbol{\Omega})$ describes those traces where the value of x increases monotonically. In general, $\mathsf{everywhere}\ \delta$ holds for traces such that for each sub-trace δ holds, and $\mathsf{somewhere}\ \delta$ for traces such that their exists a sub-trace for which δ holds. The *repetition*, $\mathsf{repeat}\ \delta$, holds for traces such that $\delta \boldsymbol{\tau} \delta \boldsymbol{\tau} \ldots$ holds (one or more repetitions of δ).

Awaiting. The *await* operator, $?\lceil \phi \rceil$, allows to wait until a state predicate becomes true. For example, $?\lceil x > 0 \rceil$ describes those runs where x is not greater then 0 except of the last state; it equals to $\lceil \neg\ x > 0 \rceil \boldsymbol{\chi} \lceil x > 0 \rceil_{=1} \vee \lceil \neg\ x > 0 \rceil_{=\infty}$. Note that the disjunction is necessary in order to allow await to deadlock; chopping implies the existence of a finite chopping point.

Choice. The *guarded choice*, $_ \to _ \oplus \ldots \oplus _ \to _$, allows to choose a behavior on bases of the validness of some temporal predicate (the guard). For example, $?\lceil x > 0 \rceil \to \boldsymbol{\Psi} x' = x - 1 \, \underline{\boldsymbol{\Omega}}_2 \oplus ?\lceil x < 0 \rceil \to \boldsymbol{\Psi} x' = x + 1 \, \underline{\boldsymbol{\Omega}}_2$ describes those traces where, after an initial segment, x is greater 0, and then is decremented by 1, or, otherwise, is less then 0 and is incremented. The choice gives priority to the path which can be first entered. If several paths can be entered at the same time, the choice uses some arbitrary but fixed priority; hence it doesn't introduces silent non-determinism.

Preemption. The *preemption*, $_ \nearrow _ \to _ \oplus \ldots \oplus _ \to _$, is another form of the choice, which allows to express that a behavior is interrupted and continued with another behavior if a certain condition becomes true. For example, the temporal predicate $\mathsf{repeat}\ (\boldsymbol{\Psi} y' = 1 \,\mathsf{div}\, x \, \underline{\boldsymbol{\Omega}}_2) \nearrow ?\lceil x = 0 \rceil \to \delta$ describes a behavior where y is repeatedly computed by dividing 1 by the value of x in the previous state; however, if at some point x becomes zero, then this behavior is interrupted and instead continued with δ.

$$\boldsymbol{\delta}\lceil\phi\rceil\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma := \{\,\sigma : \Sigma_\sim \mid \mathsf{points}\,\sigma \neq \boldsymbol{\phi} \wedge (\forall\, i : \mathsf{points}\,\sigma \bullet \langle x_1 \boldsymbol{\Delta}\,\sigma.x_1\,i, \ldots, x_n \boldsymbol{\Delta}\,\sigma.x_n\,i\rangle \in [\,\Sigma_\downarrow \mid \phi\,])\,\} \tag{2.1}$$

$$\boldsymbol{\delta\Psi}\,\phi\,\boldsymbol{\Omega\epsilon}^{\mathcal{D}}\,\Sigma := \{\,\sigma : \Sigma_\sim \mid \mathsf{points}\,\sigma \neq \boldsymbol{\phi} \wedge \mathsf{finite}\,\sigma \wedge \langle x_1 \boldsymbol{\Delta}\,\sigma.x_1\,0, x_1' \boldsymbol{\Delta}\,\sigma.x_1\,(\mathsf{last}\,\sigma), \ldots, x_n \boldsymbol{\Delta}\,\sigma.x_n\,0, x_n' \boldsymbol{\Delta}\,\sigma.x_n\,(\mathsf{last}\,\sigma)\rangle \in [\,\Delta\Sigma_\downarrow \mid \phi\,]\,\} \tag{2.2}$$

$$\boldsymbol{\delta}\delta_1\,\boldsymbol{\tau}\,\delta_2\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma := \{\,\sigma : \Sigma_\sim \mid (\exists\, i : \mathsf{points}\,\sigma \bullet \sigma[0\,..\,i] \in \boldsymbol{\delta}\delta_1\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma \wedge \sigma[i..] \in \boldsymbol{\delta}\delta_2\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma)\,\} \tag{2.3}$$

$$\boldsymbol{\delta}\ell\,\underline{R}\,e\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma := \{\,\sigma : \Sigma_\sim \mid \mathsf{finite}\,\sigma \wedge \#(\mathsf{points}\,\sigma)\,\underline{R}\,e\,\} \tag{2.4}$$

$$\boldsymbol{\delta}\mathsf{prefix}\,\delta\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma := \{\,\sigma : \Sigma_\sim \mid \exists\,\sigma' : \boldsymbol{\delta}\delta\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma \bullet \exists\, i : \mathsf{points}\,\sigma' \bullet \sigma = \sigma'[0\,..\,i]\,\} \tag{2.5}$$

$$\boldsymbol{\delta}\delta_1 \wedge \delta_2\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma := \boldsymbol{\delta}\delta_1\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma \cap \boldsymbol{\delta}\delta_2\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma \tag{2.6}$$

$$\boldsymbol{\delta}\neg\,\delta\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma := \boldsymbol{\delta}\mathit{true}\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma \setminus \boldsymbol{\delta}\delta\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma \tag{2.7}$$

$$\boldsymbol{\delta}\forall\, s \bullet \delta\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma := \{\sigma : \Sigma_\sim \mid \forall\, s \bullet (\sigma\,\mathsf{n}\,\mathit{clashes}) \in \boldsymbol{\delta}\delta\boldsymbol{\epsilon}^{\mathcal{D}}\,(\Sigma\,\mathsf{n}\,\mathit{clashes})\} \tag{2.8}$$

$$\boldsymbol{\delta}\mathit{true}\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma := \{\,\sigma : \Sigma_\sim\,\} \tag{2.9}$$

$$\boldsymbol{\delta}e_1 \in e_2\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma := \{\,\sigma : [\,\Sigma_\sim \mid e_1 \in e_2\,]\,\} \tag{2.10}$$

Fig. 1. Basic temporal predicates of $\mathcal{D}Z$

2.3 Semantics of DZ

The set constructor for dynamic values is introduced by an ordinary Z declaration, $\mathrm{dyn}\,A == \{f : \boldsymbol{\gamma}\,\boldsymbol{\kappa}\,A \mid \forall\, i : \boldsymbol{\gamma} \bullet i + 1 \in \mathsf{dom}\,f \Rightarrow i \in \mathsf{dom}\,f\}$.

We view dynamic schemata of the syntactic form $[\,x_1 : \mathrm{dyn}\,A_1;\ \ldots\ ;\ x_n : \mathrm{dyn}\,A_n \mid \delta\,]$, where constraints induced by the declarations are already shifted to the property part. The semantics of a dynamic schema is defined by the function $\boldsymbol{\delta}\delta\boldsymbol{\epsilon}^{\mathcal{D}}\,\Sigma$ (Figure 1), which translates a dynamic schema with a signature described by Σ and the property δ syntactically into a Z expression, denoting a set of bindings. We emphasize the translation view in order to demonstrate that $\mathcal{D}Z$ is syntactic sugar. For example, $\boldsymbol{\delta}\lceil x = 0\rceil\boldsymbol{\epsilon}^{\mathcal{D}}\,[x : \mathrm{dyn}\,\boldsymbol{\upsilon}]$ literally yields the set-expression $\{\sigma : [x : \mathrm{dyn}\,\boldsymbol{\upsilon}] \mid \mathsf{points}\,\sigma \neq \boldsymbol{\phi} \wedge (\forall\, i : \mathsf{points}\,\sigma \bullet \langle x \boldsymbol{\Delta}\,\sigma.x\,i\rangle \in [[x : \boldsymbol{\upsilon}] \mid x = 0])\}$.

In the definition of the translation, Σ represents a plain Z schema, $[\,x_1 : \mathrm{dyn}\,A_1;\ \ldots\ ;\ x_n : \mathrm{dyn}\,A_n \mid \phi\,]$, which is initially constructed from the declaration part of the dynamic schema to be translated. With $\Sigma_\sim$ we denote Σ extended by a property which enforces the traces of all dynamic values to be of the same length. With $\Sigma_\downarrow$ we denote the flattening of Σ, such that all declarations $x_i : \mathrm{dyn}\,A_i$ are replaced by $x_i : A_i$. $\Delta\Sigma$ is the schema operator as known from Z. The Z expression $\langle x_1 \boldsymbol{\Delta}\,e_1, \ldots, x_n \boldsymbol{\Delta}\,e_n\rangle$ denotes a binding. Given a binding $\sigma \in \Sigma_\sim$, then $\mathsf{points}\ \sigma$ yields the

$$\ell = \infty \equiv \forall\, n : \boldsymbol{\gamma} \bullet \neg\, \ell = n \tag{2.11}$$

$$\delta_{\underline{R}c} \equiv \delta \wedge \ell\ \underline{R}\ c \tag{2.12}$$

$$\delta_1\ \boldsymbol{\chi}\ \delta_2 \equiv \delta_1\ \boldsymbol{\tau}\ \mathit{true}_{=2}\ \boldsymbol{\tau}\ \delta_2 \tag{2.13}$$

$$?\lceil\phi\rceil \equiv \lceil\neg\,\phi\rceil\ \boldsymbol{\chi}\ \lceil\phi\rceil_{=1} \vee \lceil\neg\,\phi\rceil_{=\infty} \tag{2.14}$$

$$\mathsf{somewhere}\ \delta \equiv \mathit{true}\ \boldsymbol{\tau}\ \delta\ \boldsymbol{\tau}\ \mathit{true} \tag{2.15}$$

$$\mathsf{everywhere}\ \delta \equiv \neg\ \mathsf{somewhere}\ \neg\ \delta \tag{2.16}$$

$$\mathsf{repeat}\ \delta \equiv \delta \vee (\delta_{>0}\ \boldsymbol{\tau}\ \mathsf{repeat}\ \delta) \tag{2.17}$$

$$\delta_1 \to \delta_1' \oplus \delta_2 \to \delta_2' \equiv (\delta_1 \wedge \neg\,\delta_2 \wedge \mathsf{prefix}\ \delta_2)\ \boldsymbol{\tau}\ \delta_1' \vee (\delta_2 \wedge \neg\,\delta_1 \wedge \mathsf{prefix}\ \delta_1)\ \boldsymbol{\tau}\ \delta_2' \vee (\delta_1 \wedge \delta_2)\ \boldsymbol{\tau}\ \delta'_{\mathrm{oracle}} \tag{2.18}$$

$$\delta_1 \nearrow \delta_2 \to \delta_2' \equiv (\delta_1 \wedge \neg\,\delta_2 \wedge \mathsf{prefix}\ \delta_2) \vee (\mathsf{prefix}\ \delta_1 \wedge \delta_2)\ \boldsymbol{\tau}\ \delta_2' \tag{2.19}$$

Fig. 2. Derived temporal predicates of $\mathcal{D}Z$

domain of the traces of the dynamic values (which is the same for any individual dynamic value), $\mathsf{finite}\ \sigma$ tells whether the traces are finite, and $\mathsf{last}\ \sigma$ yields the last index of the traces (which is only defined if $\mathsf{finite}\ \sigma$ holds). The notation $\sigma[i\,..\,j]$ denotes the binding where all dynamic values are "sliced" from i to j, whereas $\sigma[i\,..]$ denotes the tail starting from i.

In (2.8), s represents a "schema text" (a set of declarations). In our translation view, we just insert this text in the resulting expression. However, names introduced by s can shadow names from Σ; we have expressed this only informally by the phrase $\sigma\ \mathsf{n}\ \mathit{clashes}$ resp. $\Sigma\ \mathsf{n}\ \mathit{clashes}$, which remove components with clashing names from a binding resp. hide them in a schema. In (2.4), the expression e is just syntactically inserted; it cannot depend on names from Σ. In (2.10), by using $\Sigma_\sim$ in the position of schema inclusion, its declarations are bounded for e_1 and e_2.

In (2.5), an auxiliary predicate is introduced, which is not supposed to be visible on the user level, but allows to describe preemption and choice as a derived operator: $\mathsf{prefix}\ \delta$ holds for those traces which are a prefix of the traces for which δ holds.

The derived forms of temporal predicates are summarized in Figure 2. We present only the special case of choice with two paths and preemption with one watchdog. In (2.18), the condition $\neg\,\delta_i \wedge \mathsf{prefix}\ \delta_i$ ensures that the guard of the other path is still "running" when a path is taken. From an operational view, the choice executes all its guards in parallel, and the path with the first guard which proceeds wins. In case that both ones can proceed at the same time, δ'_{oracle} defines that a fixed but arbitrary path δ_1' or δ_2' is taken; hence the choice does not introduce silent moves.

$$SharedState[A] \mathrel{\mathbb{b}} [val : A;\ writers : \alpha\,\mathrm{place}]$$
$$\mathrm{shared}\ A == \{\, x : \mathrm{dyn}(SharedState[A]) \mid \lceil \#(x.writers) \leq 1 \rceil \wedge \mathsf{everywhere}\ (\ell = 2 \Rightarrow \Psi(x'.writers = \phi \Rightarrow x'.val = x.val)) \}$$
$$_:=_\,_[A] == \{x : SharedState[A];\ p : \mathrm{place};\ a : A \mid \{p\} = x.writers \wedge x.val = a \vee \{p\} \neq x.writers \supset \phi\}$$

Fig. 3. Definition of Shared Variables in $\mathcal{D}Z$

3 Concurrency

Concurrency can be characterized as a situation where distinguishable *activities* compete for the access to restricted *resources*. Such behavior is modeled in $\mathcal{D}Z$ without extending the language. We will illustrate this by the example of *shared variables* with *racing semantics*. With *racing* we denote the situation where two activities write to a shared variable at the same time; it is resolved in that only one write succeeds whereas the other is ignored. Racing already entails everything necessary to study the problem of concurrency in the framework of $\mathcal{D}Z$; more high-level concepts such as synchronous channels do not introduce additional problems.

In order to identify concurrent activities, we introduce the notion of a so-called *place*. A place is a "point in space" that identifies a concurrent activity. Places are represented in $\mathcal{D}Z$ by the given set place.

Shared variables are represented by a subset of dyn($[val : A;\ writers : \alpha\,\mathrm{place}]$) (Figure 3). The field *val* represents the current value and the field *writers* the set of places writing to the shared variable. Every shared variable satisfies the constraint that there is never more then one writer and the value keeps persistent during writes. This invariant is defined by using dynamic set comprehension for the declaration of shared A.

The relation $x :=_{\mathsf{p}} y$ models a write to a shared value x from place p with value y. A concurrent write operation is logically expressed as $\lceil x :=_{\mathsf{p}} a \rceil \wedge \lceil x :=_{\mathsf{q}} b \rceil$, with $p \neq q$, which is satisfied for those traces such that x has either the value a or the value b (provided that no other places then p and q do write).

Though we have defined what happens if a write action is performed, we have not described yet how unintended writes are *excluded*. Unwanted behavior has to be specified explicitly in an axiomatic framework – this is similar to plain Z where operation schemata have to include equalities such as $x' = x$ if x is supposed to keep its value. To model which places write or write not to a shared variable x, we use predicates such

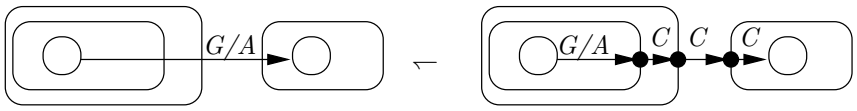


Fig. 4. Remove Inter-level Transitions

as $\lceil x.writers \cap ps = \phi \rceil$, where ps is a set of places that shall not write to x during a trace. The usage of such constraints will be tackled in more detail in the next Section.

4 Translating Statecharts to $\mathcal{DZ}$

Statecharts [6] are a graphical notation for hierarchical structured automatons, which have gained relative relevance in industrial applications. Statecharts and their semantics have been subject to a notably amount of research activities, leading to various syntactic and semantic variants (see [16]). Since the ESPRESS project aims at a tool environment which is centered around the Statemate tool [7], we consider the Statecharts semantics as implemented by Statemate and presented in [8].

4.1 Dealing with Inter-Level Transitions

The pathological problem when mapping Statecharts to some text-based language is to cope with the problem of inter-level transitions. These transitions cross several hierarchy levels, by leaving not only the direct source state but also enclosing super-states (a concept similar to exceptions), or by jumping directly into a sub-state of another state (a concept which has no direct counterpart in structured programming).

To cope with this problem, we normalize Statecharts prior to translation. The normalization cuts the transition at points where it crosses level-borders. At the cut-points, *connectors* are introduced, which are used to reconstruct inter-level transitions when composing states from their sub-states. This concept is similar to the *Unvollendete* introduced in [10]. Figure 4 shows an example. The transition labeled with G/A is partitioned into three sub-transitions, where the guard is assigned to the first transition. The connectors are all labeled with the same name C, which uniquely identifies them belonging to the same compound transition.

4.2 Outline of the Translation

Assuming the sketched normalization, we can treat the translation of states independent from their context. The basic strategy is illustrated

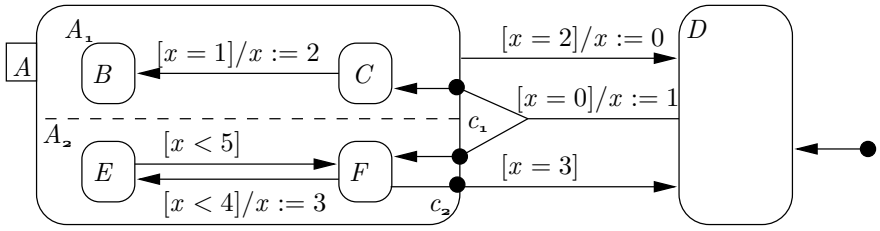


Fig. 5. Example Statechart

by the Statechart in Figure 5, whose translation is given in Figure 6. Each state is represented by exactly one dynamic schema. Due to loops in the Statechart, the schema definitions are recursive. Nevertheless, the construction creates only tail-recursion, which can be replaced by terms build from the repeat operator. From the structure of the Statechart, additional declarations are induced. The schema Σ contains the data variables of the chart. In addition to the variable x, it declares event variables c_i which are used to realize the gluing of inter-level-transitions: whenever the first sub-transition of the cut of the transition is taken, the associated event c_i is generated; the remaining parts of the cut are guarded with the presence of this event, and will fire instantaneously until the target state is reached. Event variables themselves are represented as shared variables (Section 3) of a special type $trigger ::= \bot$ that has only one value ($_E == \text{shared } trigger$). $e?$ tests whether the event e is present ($e? \equiv e.writers \neq \phi$) and $e!_p$ emits it ($e!_p \equiv e :=_p \bot$), where p denotes a place.

In order to represent concurrent activities of the chart, the translation introduces for each state S that is a sub-state of an xor-state, a place S_p. Note that sub-states of an and-state (i.e. A_1, A_2) do not write variables and therefore do not need places. From the structure of the Statechart, we define the function *exclusive*, statically mapping each state-place to the set of state-places which definitively can not write when the given state is active, i.e. its neighboring states and their sub-states. This information can be used to derive whether and with whom racing can happen. The "schematic" schemata $NoWrite_p$ and $Write_{p,vs}$ are used to model effects on variables during the activity of a state with place p. Besides resolving write conflicts, *Write* and *NoWrite* also prevent unintended writes.

The sketched translation is rather naive, since it does not perform any "optimizations" of the use of shared variables. Such an optimization can easily derive on base of the represented model, for which write operations racing is excluded, and can replace in such cases $x :=_p v$ by the predicate $x.val = v \wedge x.writers = \{p\}$. In the shown example, this could be done

$$
\begin{array}{l}
\mathit{Init} \mathbin{\text{-}\!\!\text{b}} D \\
A \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid (\mathit{NoWrite}_{\mathrm{A}\mid} \wedge ?\lceil c_1 ?\rceil \to (A_1 \wedge A_2)) \nearrow \\
\qquad\qquad ?\lceil c_2 ?\rceil \to D \oplus ?\lceil x.\mathit{val} = 2\rceil \to (\mathit{Write}_{\mathrm{p},\{\mathrm{x}\}} \wedge \Psi\, x' :=_{\mathrm{A}\mid}\, 0 \,\Omega_{-2})\ \tau\, D\,] \\
A_1 \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid ?\lceil c_2 ?\rceil \to C\,] \qquad A_2 \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid ?\lceil c_2 ?\rceil \to F\,] \\
B \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid \mathit{NoWrite}_{\mathrm{B}\mid}\,] \\
C \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid \mathit{NoWrite}_{\mathrm{C}\mid} \nearrow ?\lceil x.\mathit{val} = 1\rceil \to (\mathit{Write}_{\mathrm{C}\mid,\{\mathrm{x}\}} \wedge \Psi\, x' :=_{\mathrm{C}\mid}\, 2 \,\Omega_{-2})\ \tau\, B\,] \\
E \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid \mathit{NoWrite}_{\mathrm{E}\mid} \nearrow ?\lceil x.\mathit{val} < 5\rceil \to (\mathit{NoWrite}_{\mathrm{E}\mid} \wedge \Psi\, \mathit{true}\, \Omega_2\ \tau\, F\,] \\
F \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid \mathit{NoWrite}_{\mathrm{F}\mid} \nearrow ?\lceil x.\mathit{val} < 4\rceil \to (\mathit{Write}_{\mathrm{F}\mid,\{\mathrm{x}\}} \wedge \Psi\, x :=_{\mathrm{F}\mid}\, 3 \,\Omega_{-2})\ \tau\, E \\
\qquad\qquad \oplus ?\lceil x.\mathit{val} = 2\rceil \to (\mathit{Write}_{\mathrm{C}\mid,\{\mathrm{c}_2\}} \wedge \Psi\, \not{c}_2!_{\mathrm{F}\mid}\, \Omega_{-2}))\,] \\
D \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid \mathit{NoWrite}_{\mathrm{D}\mid} \nearrow ?\lceil x.\mathit{val} = 0\rceil \to (\mathit{Write}_{\mathrm{D}\mid,\{\mathrm{x},\mathrm{c}_1\}} \wedge \Psi\, x' :=_{\mathrm{D}\mid}\, 1 \wedge c_1'!_{\mathrm{D}\mid}\, \Omega_{-2})\ \tau\, A\,] \\
\\
\Sigma \mathbin{\text{-}\!\!\text{b}} [\, x : \mathrm{shared}\, \upsilon;\ c_1, c_2 : \mathrm{E}\,] \\
\mathit{place} ::= A_{\mathrm{p}} \mid B_{\mathrm{p}} \mid C_{\mathrm{p}} \mid D_{\mathrm{p}} \mid E_{\mathrm{p}} \mid F_{\mathrm{p}}; \\
\mathit{exclusive} == \{A_{\mathrm{p}} \mapsto \{D_{\mathrm{p}}\}, B_{\mathrm{p}} \mapsto \{C_{\mathrm{p}}\}, C_{\mathrm{p}} \mapsto \{B_{\mathrm{p}}\}, \\
\qquad\qquad D_{\mathrm{p}} \mapsto \{A_{\mathrm{p}}, B_{\mathrm{p}}, C_{\mathrm{p}}, E_{\mathrm{p}}, F_{\mathrm{p}}\}, E_{\mathrm{p}} \mapsto \{F_{\mathrm{p}}\}, F_{\mathrm{p}} \mapsto \{E_{\mathrm{p}}\}\} \\
\mathit{NoWrite}_{\mathrm{p}} \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid (\mathrm{each}\ v : \{x, c_1, c_2\} \bullet \lceil (\{p\} \cup \mathit{exclusive}\ p) \cap v.\mathit{writers} = \phi\rceil)\,] \\
\mathit{Write}_{\mathrm{p},\mathrm{vs}} \mathbin{\text{-}\!\!\text{b}} [\, \Sigma \mid \mathit{NoWrite}_{\mathrm{p}}\ \chi\, ((\mathrm{each}\ v : \{x, c_1, c_2\} \setminus \mathit{vs} \bullet \lceil p \notin v.\mathit{writers}\rceil) \wedge \\
\qquad\qquad (\mathrm{each}\ v : \mathit{vs} \bullet \lceil \mathit{exclusive}\ p \cap v.\mathit{writers} = \phi\rceil))_{=1}\,]
\end{array}
$$

Fig. 6. Naive Translation of the example Statechart

for the events and transition going from A to D and vice versa. More sophisticated techniques would base on theorem proving and take exclusive guards into consideration. A complete translation from Statemate Statecharts to $\mathcal{DZ}$ is out of scope of this paper. It can be found in an extended version.

5 Conclusion

Discussion and Related Work. We have presented $\mathcal{DZ}$, a framework which combines Z and discrete temporal interval logics for the formalization of properties as well as of behaviors of embedded systems. $\mathcal{DZ}$ is a moderate extension of Z, since it adds only a few new syntactic constructs, which can be explained as syntactic sugar for existing concepts of Z. As a consequence, we could relative easily extend the ESPRESS Z type checker, *ESZ*, to support $\mathcal{DZ}$[2]. Our integration of Z with interval logics differs from similar approaches which base e.g. on the duration calculus [3,5], since we focus on a discrete time model, and support a tailored set of

[2] The *ESZ* type checker is downwards compatible to fUZZ, and will be become available in the middle of '97.

operators for the operational description of behavior, which is oriented towards notions of process algebras [9, e.g.]. We are currently restricted to external choice. By that, we can represent the semantics as a set of traces and avoid problems arising with silent moves. The restriction to a trace semantics also allows to mix "don't care" non-determinism as introduced by specification looseness and "don't know" non-determinism as a result of behavioral descriptions.

The Statechart translation yields a setting, where (theoretically) we can argue about propositions such as a Statechart implies a property given by a trace invariant. We hope that by extending an existing encoding of Z in a theorem prover such as Isabelle [11], it is possible to provide tool support for such propositions. Extending an encoding of Z to $\mathcal{D}Z$ should not be to hard, since $\mathcal{D}Z$ is a moderate extension of Z. However, developing the set of adequate theorems, to make such an encoding of $\mathcal{D}Z$ practical, will need a lot of further work.

As a side-effect, the translation of Statecharts provides a formalization of the semantics of Statecharts as found in the Statemate tool [7]. A main characteristic of this semantics is that actions that are performed in one step will not become visible before the next step, thereby avoiding the problem of global inconsistency. In contrast, the semantics presented e.g. in [14,10] allow transitions to immediately react upon emited events. To our knowledge, the semantics we have given is among the first formalizations of the Statemate version of Statecharts [4,12].

Future Work. $\mathcal{D}Z$ needs to be evaluated in case studies, which is currently done in the course of the Espress project. These experiments shall give evidence about the applicability of discrete temporal interval logics for the specification of properties of embedded systems. We are currently studying an extension of $\mathcal{D}Z$ to the hybrid case, which bases on the model of phase transition systems. We need to further investigate if and how to represent internal choice; one possible direction is to adapt the notion of failure sets [9] to the framework of interval logics. Related is the problem of distinguishing "don't care" from "don't know" indeterminism, which probably needs to be done in the presence of internal choice. Last not least, we plan to extend the encoding of Z in Isabelle used in Espress [11] to the case of $\mathcal{D}Z$, on the one hand to validate the model itself and to derive useful theorems, on the other hand to open possibilities for deduction based tool-support.

Acknowledgements. We would like to thank Frank Lattemann and Eckard Lehmann for the fruitful discussions on Z and interval logics, and Robert Geisler and Marcus Klar for the discussions about Z and Statecharts.

References

1. R. Büssow, R. Geisler, and M.Klar. Spezifikation eingebetteter Steuerungssysteme mit Z und Statecharts. In *Tagungsband zur 5. Fachtagung Entwurf komplexer Automatisierung ssysteme*. TU Braunschweig, 1997.
2. Robert Büssow, Heiko Dörr, Robert Geisler, Wolfgang Grieskamp, and Marcus Klar. μSZ – ein Ansatz zur systematischen Verbindung von Z und Statecharts. Technical Report TR 96-32, Technische Universität Berlin, 1996.
3. Z. Chaochen, C. A. R. Hoare, and A.P Ravn. A calculus of durations. *Information Processing Letters*, 40(5), 1991.
4. Hartmut Ehrig, Robert Geisler, Marcus Klar, and Julia Padberg. Horizontal and vertical structuring techniques for statecharts. In *Proc. CONCUR'97*, Lecture Notes in Computer Science. Springer-Verlag, Berlin, Heidelberg, New York, 1997.
5. M. Engel. Specifying real-time systems with Z and the Duration Calculus. In J. Bowen and J. Hall, editors, *Z User Workshop, Cambridge 1994*, Workshops in Computing, pages 282–294. Springer-Verlag, Berlin, Heidelberg, New York, 1994.
6. David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, June 1987.
7. David Harel, Hagi Lachover, Amnon Naamad, Amir Pnueli, Michal Politi, Rivi Sherman, Aharon Shtull-Trauring, and Mark Trakhtenbrot. Statemate: A working environment for the development of complex reactive systems. *IEEE Transactions on Software Engineering*, 16 No. 4, April 1990.
8. David Harel and Ammon Naamad. The statemate semantics of statecharts. Technical report, The Weizmann Institute of Science, October 1995.
9. C.A.R. Hoare. *Communicating Sequential Processes*. Prentice Hall, Eaglewood Cliffs, N.J., 1985.
10. C. Huizing, R. Gerth, and W. P. de Roever. Modelling statecharts behaviour in a fully abstract way. In *Proc. 13th CAAP*, volume 299 of *Lecture Notes in Computer Science*. Springer-Verlag, 1988.
11. Kolyang, Thomas Santen, and Burkhart Wolff. A structure preserving encoding of Z in Isabelle/HOL. In J. von Wright, J. Grundy, and J. Harrison, editors, *Theorem Proving in Higher-Order Logics*, number 1125 in Lecture Notes in Computer Science. Springer-Verlag, Berlin, Heidelberg, New York, 1996.
12. E. Mikk, Y. Lakhnech, C. Petersohn, and M. Siegel. On formal semantics of statecharts as supported by statemate. In *The Nothern Formal Method Workshop Series*. Springer Verlag, 1996.
13. B. C. Moszkowski. *Reasoning about Digital Circuits*. PhD thesis, Stanford University, 1983. Tech. Report STAN-CS-83-970.
14. A. Pnueli and M. Shalev. What is in a step: On the semantics of statecharts. In Takayasu Ito and Albert R. Meyer, editors, *Theoretical Aspects of Computer Software*, pages 244–264, New York, September 1991. Springer-Verlag. Lecture Notes in Computer Science 526.
15. J. M. Spivey. *The Z Notation: A Reference Manual*. Prentice Hall International Series in Computer Science, 2nd edition, 1992.
16. Michael von der Beeck. A comparison of statecharts variants. In Langmaak, de Roever, and Vytopil, editors, *Formal Techniques in Real-Time and Fault-Tolerant Systems*, volume 863 of *Lecture Notes in Computer Science*, pages 128–148, 1994.